

4

Assembling the Context Engine

In the previous chapters, we learned how to engineer context and build agentic systems from the ground up. But as those systems grow to have more agents, more responsibilities, more moving parts, they become increasingly difficult to manage. That's where the **Context Engine** comes in.

Think of the Context Engine as not just another piece of software but a structured orchestration layer, a thinking workflow designed to turn a vague idea, like a simple user request, into a fully formed, context-aware output. Think of it as the conductor of your agent orchestra. It doesn't perform the tasks itself; instead, it coordinates, directs, and connects each specialist so they work in perfect sync.

In this chapter, we'll start with a high-level look at the Context Engine's architecture, then dive into the functions that bring it to life. You'll see that at its core, the engine follows a simple yet powerful rhythm: **plan, execute, and reflect**. By the end, you'll not only understand how the Context Engine works but you'll have built one yourself. You'll see how to assemble its components step by step, how to connect it to your agents using MCP, and how to design a reasoning system that's scalable, transparent, and resilient.

In a nutshell, this chapter covers the following topics:

- Architecting the Context Engine
- Assembling the system
- Running the engine

Let's begin by defining the architecture of the Context Engine.

Architecting the Context Engine

The evolution of artificial intelligence has moved from rigid, rule-based systems to flexible, generative models. Think of early AI, often called symbolic AI, as carefully hand-crafted rule books. They were impressive in narrow domains, yet brittle in the face of ambiguity and anything outside the script. Modern LLMs flip that pattern. They show broad language understanding and general reasoning, but they lack specific, timely knowledge, and they cannot reliably act on external systems. On their own, LLMs rarely deliver sustained business value. This gap led to two shifts you have already seen in the previous chapters. First, RAG grounds LLMs with factual context from external sources, which reduces hallucinations and makes responses situationally aware. Second, AI agents give LLMs tools and a measure of autonomy so they can execute multi-step tasks. Together, these ideas nudge models closer to useful work.

However, this rapid evolution has created a new architectural challenge. Early agentic systems often hardcoded the handoffs between agents, which feels fine for a demo and quickly becomes a tangle in production. If you have wired more than two agents together, you know this pain. As

teams add specialized agents, the linear assembly line cannot reason about its own capabilities or adapt to novel problems. So, the question becomes: how do we move from a collection of agents to an adaptive, autonomous system? That is where the Context Engine comes in.

The Context Engine is a meta-system, an intelligent controller that organizes specialized agents around a high-level goal. It is more than a single function; it is a structured workflow that turns a vague intent into a grounded, context-aware output. While our label “Context Engine” is specific to this framework, the pattern is widely used in advanced generative AI platforms. Systems such as ChatGPT or Gemini are not single models in isolation. They are applications with controllers that manage sessions, orchestrate tools, and maintain context. Conceptually, those controllers play the same role as a Context Engine.

The uniqueness of the architecture lies in its explicit separation of planning from execution, inspired by cutting-edge research in agentic reasoning. Rather than a simple loop, the Context Engine employs a two-phase process:

- First, a **Planner** acts as the strategic core, reasoning about the user's goal in using the **Agent Registry** as a “toolkit” of available capabilities to select the best agent or function. We will thus build these new additional **Planner** and **Agent Registry** tools in this chapter.
- Then, the **Planner** uses an LLM to generate a dynamic, step-by-step execution plan tailored to the specific task. Once the plan is ready, it is handed off to an **Executor**, the operational manager who invokes the specialist agents in the correct sequence.

As introduced in [Chapter 3](#), we will leverage dual RAG with static knowledge data and dynamic instructions to enhance the flexibility of the Context Engine.

In this section, we will design the Context Engine. We will begin with a visual walk-through of the architecture and then unpack the components it relies on. See [Figure 4.1](#) for the overview.

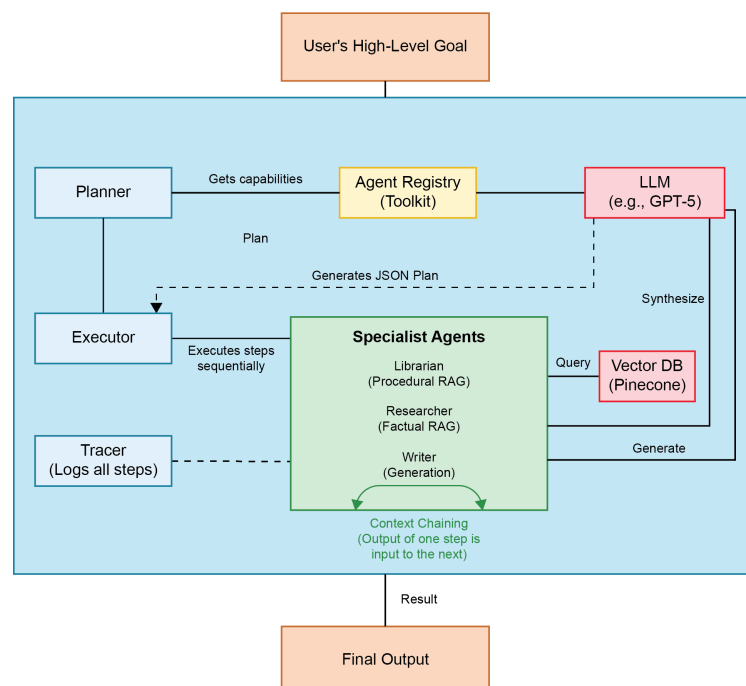


Figure 4.1: The architecture of the Context Engine

Architectural overview

The Context Engine is a dynamic, multi-stage workflow, as shown in the diagram. Each color represents a distinct functional layer that collaborates to transform a high-level goal into a fully generated output.

We begin at the outermost layer, where the process starts and ends with the user. The orange components represent these two touchpoints: the **User's High-Level Goal** and the **Final Output**. The goal is a broad instruction like "Write a suspenseful story". It's a simple request without any predefined structure. The final output, on the other hand, is the complete, polished result that fulfills this goal.

At the center of it all lies the **Context Engine** itself, represented in blue. This is the orchestrator, the brain that doesn't perform the tasks directly but manages the entire workflow with strategic intelligence. Inside it are three key modules:

1. **Planner**: The strategic core that receives the user's goal and designs a step-by-step plan.
2. **Executor**: The operational manager that carries out the plan, calling specialized agents and managing the data flow between them.
3. **Tracer**: The transparent recorder that logs every action for debugging and insight.

The first phase in this workflow is **strategic planning**, where the Planner collaborates with the yellow and red components. It starts by consulting the yellow-highlighted **Agent Registry**, a toolkit that lists all available agents and their capabilities. With this information, it contacts an external red-highlighted service, the **LLM** (e.g., GPT-5). The Planner sends both the user's goal and the toolkit description to the LLM, which then returns a structured JSON plan. This plan, shown as a dashed arrow in the diagram, flows back into the engine and is handed over to the Executor.

The second phase is **execution**, where the Executor coordinates the green **Specialist Agents** and external red services. Following the plan step by step, it calls on the following:

1. **The Librarian** handles procedural RAG to fetch stylistic blueprints.
2. **The Researcher** performs factual RAG to gather and synthesize relevant information.
3. **The Writer** combines style and facts to produce the final content.

Each of these agents interacts with external services. The Librarian and Researcher query the **Vector DB**, while the Researcher and Writer may call the LLM to refine facts or generate text.

A key innovation here is context chaining (introduced in *Chapter 1*), shown by the looping green arrow. This mechanism ensures a stateful workflow, where the output of one agent (say, the blueprint of the Librarian) becomes the input for the next (the Writer).

With this overview in place, we're now ready to explore the specific functions that bring the Context Engine to life.

Functional deep dive

Now let's see how the functions in the notebook bring the architecture in *Figure 4.1* to life. The core helper functions handle the basic tasks as follows:

- `call_llm_robust()` is an interface to the LLM, used by the Planner and agents to communicate with the OpenAI API.
- `get_embedding()` converts text into vectors for the Pinecone vector store. It is used to store data and also embed requests to query Pinecone.
- `query_pinecone()` allows agents to query the Pinecone vector store.
- `create_mcp_message()` is the standardized communication protocol for Executor-agent interactions.
- `resolve_dependencies()` is the core of context chaining, wiring agent inputs and using outputs for the next task in the context chaining process.

We will also need specialist agents (green components):

- `agent_context_librarian()` fetches dynamic semantic instruction blueprints
- `agent_researcher()` fetches static data and synthesizes facts.
- `agent_writer()` generates the final content.

The system then requires orchestration and toolkit functions (the blue and yellow components):

- `AgentRegistry` initiates the Agent Registry
- `get_handler()` retrieves the agent function for the Executor to perform specific tasks
- `get_capabilities_description()` provides the agent list to the Planner
- `planner()` powers the Planner, orchestrating interactions between the Agent Registry and LLM to create the plan
- `context_engine()` is the main orchestrator function, running the Planner and Executor in sequence
- `ExecutionTrace` implements the Tracer, logging every step for debugging

The functions mirror the diagram's arrows:

1. User request → calls `context_engine()`.
2. Inside the engine → `planner()` is called first. It uses `AgentRegistry.get_capabilities_description()` and `call_llm_robust()` to generate a plan.
3. The Executor iterates through the plan, using `resolve_dependencies()` to assemble context for the current agent.
4. The Executor retrieves the correct agent using `AgentRegistry.get_handler()` (e.g., `agent_researcher()`) and invokes it.
5. Agents use helper functions:
 1. Librarian and Researcher → `query_pinecone()` (uses `get_embedding()`)
 2. Researcher and Writer → `call_llm_robust()`

Each agent returns a result with `create_mcp_message()`, which the Executor stores for the next step. The cycle continues until the plan is fully completed and the final output is returned. With that, we have gone through the entire architecture. Now, let's build the system!

Assembling the system

We've spent time understanding how the Context Engine is structured and how each component contributes to the larger workflow. In this section, we shift from understanding to building. Our focus now moves from the architectural blueprint to the running system itself. The `Context_Engine.ipynb` notebook will be our workspace, the environment where we will assemble the engine piece by piece, connecting the gears and wiring the circuits that bring our diagram to life.

We'll build the engine in a logical order, starting from the foundation upward. A capable orchestrator is only as effective as the team it leads, so our first task is to create the **specialist agents**, the functional core of the system. Once our agents are ready, we'll develop their **toolkit**, the Agent Registry, which makes their capabilities discoverable and easy to manage. Finally, we'll assemble the **Planner** and **Executor**, the coordinating intelligence that turns all the parts into a unified operation. We begin with the specialist agents!

A note on latency: The Context Engine built in this book and the accompanying repository performs complex, multi-step reasoning, not simple, single-shot answers. The delay you observe in Colab is the *thinking* time, as the engine dynamically plans and executes a sequence of API calls (e.g., planning, then RAG, then generation). This is the same reason advanced platforms like Gemini or ChatGPT require a moment to *think* for complex requests.

The specialist agents

Before we can bring the Context Engine fully to life, we need to build its front-line specialists—the agents that actually do the work. As we've already established, the multi-agent system is composed of three specialized agents: Librarian, Researcher, and Writer. Each agent serves a distinct purpose, and all agents communicate using MCP, which acts as a universal language or a standardized "shipping container" for information moving through the system. Without a shared standard, each agent would require custom integration logic, making the system brittle and difficult to scale. The MCP solves this by ensuring that every piece of data, whether it's a stylistic blueprint from the Librarian or a factual summary from the Researcher, is packaged in the exact same format.

This standardization is what enables **context chaining**. Each agent receives an MCP message as input and returns another as output, allowing the Executor to simply pass along the data without worrying about translation or formatting. This seamless flow of information is what makes stateful, multi-step reasoning possible and is the foundation upon which the entire engine operates.

Context Librarian agent

Our first agent is the Context Librarian, implemented as `agent_context-librarian`. Its role is to identify the user's intent and fetch the appropriate semantic blueprint from the vector store. As we explained earlier, this blueprint provides instructions for the Writer on how to structure and style the content.

The Librarian begins by extracting an `intent_query` from the MCP message, something like *"suspenseful narrative blueprint."* This is the critical first moment of interpretation, where a broad creative idea is turned into something machine-readable. It deciphers the desired *feeling or format* from the high-level request, translating a broad concept like "suspense" into a precise, actionable search query for its knowledge base:

```
def agent_context_librarian(mcp_message):  
    """  
    Retrieves the appropriate Semantic Blueprint from the Context Library.
```

```
"""
print("\n[Librarian] Activated. Analyzing intent...")
requested_intent = mcp_message['content'].get('intent_query')

if not requested_intent:
    raise ValueError("Librarian requires 'intent_query' in the input content.")
```

The Librarian then performs a semantic search in the context vector store using `query_pinecone()`. Think of this vector store not as a factual database but as a *library of recipes*, each blueprint describing how a particular kind of writing should be structured or styled. The search isn't just matching words; it's matching *meaning*, allowing the system to find the closest conceptual fit even if the phrasing doesn't match exactly:

```
results = query_pinecone(requested_intent, NAMESPACE_CONTEXT, top_k=1)
```

If a blueprint is found, it is packaged into a new MCP message; otherwise, the Librarian still returns a valid message, just one with generic instructions:

```
if results:
    match = results[0]
    print(f"[Librarian] Found blueprint '{match['id']}' (Score: {match['score']:.2f})"
    blueprint_json = match['metadata']['blueprint_json']
    content = blueprint_json
else:
    print("[Librarian] No specific blueprint found. Returning default.")
    content = json.dumps({"instruction": "Generate the content neutrally."})

return create_mcp_message("Librarian", content)
```

This final step demonstrates the system's resilience. In an ideal scenario, a perfectly matched, expert-crafted blueprint is found and sent downstream to the Writer, ensuring a highly tailored output. However, if no relevant blueprint exists, the system doesn't fail. It gracefully defaults to a neutral instruction, ensuring that the workflow can always be completed, albeit with a more generic result.

Researcher agent

Next comes the **Researcher**, the system's investigative journalist. While the Librarian handles structure and tone, the Researcher grounds everything in fact. Its job is to query the knowledge vector store, retrieve the most relevant information for a given topic, and synthesize it into a concise factual summary.

Let's walk through how this agent turns a broad topic into a focused, verifiable brief. The process begins when the agent receives its assignment, the `topic_query`, from the incoming MCP message. Its first action is to perform a semantic search of the `NAMESPACE_KNOWLEDGE` vector store, retrieving the `top_k=3` most relevant chunks of text:

```
def agent_researcher(mcp_message):
    """
    Retrieves and synthesizes factual information from the Knowledge Base.
    """
    print("\n[Researcher] Activated. Investigating topic...")
```

```

topic = mcp_message['content'].get('topic_query')

if not topic:
    raise ValueError("Researcher requires 'topic_query' in the input content.")

results = query_pinecone(topic, NAMESPACE_KNOWLEDGE, top_k=3)

if not results:
    print("[Researcher] No relevant information found.")
    return create_mcp_message("Researcher", "No data found on the topic.")

print(f"[Researcher] Found {len(results)} relevant chunks. Synthesizing...")
source_texts = [match['metadata']['text'] for match in results]

```

This is a crucial design choice: by retrieving multiple sources, the agent gathers a more comprehensive and nuanced set of information than a single result could provide.

However, the agent's most important work happens in the synthesis phase. It does not simply pass along the raw, retrieved text. Instead, it uses a carefully engineered prompt to instruct an LLM to act as an `expert research synthesis AI`.

The `system_prompt` sets strict guardrails, commanding the LLM to `Focus strictly on the facts provided and not add outside information`:

```

system_prompt = """You are an expert research synthesis AI.
Synthesize the provided source texts into a concise, bullet-pointed summary relevant t

```

This is a critical step in mitigating hallucinations and ensuring the final summary is grounded in real, retrieved data. The `user_prompt` complements this by assembling the retrieved text chunks into a clean, structured format for the LLM to process:

```

user_prompt = f"Topic: {topic}\n\nSources:\n" + "\n\n---\n\n".join(source_texts)

findings = call_llm_robust(system_prompt, user_prompt)

return create_mcp_message("Researcher", findings)

```

The final synthesized findings represent a new piece of knowledge that's distilled, concise, and directly relevant to the initial topic. The output is returned in an MCP message, ready for downstream agents. This two-step retrieve-and-synthesize process ensures that the Context Engine's final output is both well-structured (thanks to the Librarian) and factually reliable.

Writer agent

The Writer agent is the final specialist in our assembly line; it's the master craftsman who combines the blueprint of the Librarian with the findings of the Researcher to generate the final text. It is responsible for the ultimate synthesis, taking the abstract style and raw facts and weaving them into a polished, coherent piece of content. It can also handle rewriting tasks, using previously generated content as input, which gives the engine the ability to iteratively refine its own work.

Let's explore how this agent skillfully assembles its inputs to produce the final output. The agent first gathers its raw materials from the MCP message: the blueprint from the Librarian, the synthesized facts from the Researcher, or any `previous_content` from prior writing steps:

```
def agent_writer(mcp_message):
    """
    Combines the factual research with the semantic blueprint to generate the final output
    Crucially enhanced to handle either raw facts OR previous content for rewriting tasks.
    """
    print("\n[Writer] Activated. Applying blueprint to source material...")

    blueprint_json_string = mcp_message['content'].get('blueprint')
    facts = mcp_message['content'].get('facts')
    previous_content = mcp_message['content'].get('previous_content')
```

Next comes a crucial check: the Writer cannot proceed without a blueprint that defines *how* the text should be written:

```
if not blueprint_json_string:
    raise ValueError("Writer requires 'blueprint' in the input content.")
```

Once the blueprint is verified, the Writer determines the type of task it's performing through the `if/elif/else` block: is it creating new content from research, or is it rewriting an existing piece?

```
if facts:
    source_material = facts
    source_label = "RESEARCH FINDINGS"
elif previous_content:
    source_material = previous_content
    source_label = "PREVIOUS CONTENT (For Rewriting)"
else:
    raise ValueError("Writer requires either 'facts' or 'previous_content'.")
```

This conditional logic makes the Writer flexible. It can shift between modes (new generation or rewriting) without changing its core design. In larger workflows, this flexibility means the same agent can be reused for multiple stages of refinement.

Once the correct input is selected, the Writer constructs two prompts: a system prompt that defines how to write, and a user prompt that defines *what* to write about:

```
# The Writer's System Prompt incorporates the dynamically retrieved blueprint
system_prompt = f"""You are an expert content generation AI.
Your task is to generate content based on the provided SOURCE MATERIAL.
Crucially, you MUST structure, style, and constrain your output according to the rules

--- SEMANTIC BLUEPRINT (JSON) ---
{blueprint_json_string}
--- END SEMANTIC BLUEPRINT ---

Adhere strictly to the blueprint's instructions, style guides, and goals. The blueprint
"""

user_prompt = f"""
--- SOURCE MATERIAL ({source_label}) ---
```



```
{source_material}
--- END SOURCE MATERIAL ---

Generate the content now, following the blueprint precisely.
"""
```

This separation of prompts is where the true intelligence of the Writer lies. The system prompt provides the *creative framework*, the stylistic and structural constraints drawn from the blueprint. The user prompt supplies the *substance*, the facts or existing content that the LLM must build upon.

By separating the *how* from the *what*, the Writer ensures clarity for the model. The LLM knows exactly what it's being asked to do and under what constraints, producing outputs that are both stylistically consistent and factually grounded.

Finally, the Writer sends the prompts to the LLM, captures the result, and packages it into a standardized MCP message for downstream use:

```
final_output = call_llm_robust(system_prompt, user_prompt)
return create_mcp_message("Writer", final_output)
```

The output at this stage is the culmination of the entire Context Engine workflow. It's a piece of content supported by evidence and delivered with stylistic precision.

Agent Registry

Now that we've built our team of specialist agents, the next question is, who keeps track of them all? The Planner can't just guess which agents exist or what they're capable of. It needs a reliable directory that lists every available specialist, their role, and how to call them. To solve this, we introduce the Agent Registry: a central directory that turns a collection of agents into a coordinated team.

The `AgentRegistry` class manages all agents and their capabilities, making them discoverable for the Planner. Let's take a look at how it's structured.

The `__init__` method defines a dictionary called `self.registry`, a functional roster of all available agents. Each human-readable name, such as "Librarian," maps directly to its corresponding Python function, such as `agent_context_librarian`. This design makes the system flexible. If new agents are added later, say a "Critic" or "Editor," they can simply be registered here without changing any other part of the code base:

```
class AgentRegistry:
    def __init__(self):
        self.registry = {
            "Librarian": agent_context_librarian,
            "Researcher": agent_researcher,
            "Writer": agent_writer,
        }
```

Next comes the method that lets other components, such as the Executor, retrieve these agents on demand. The `get_handler()` method is how the Executor interfaces with the registry. When the Executor needs to run a specific step in the plan, it calls something like `get_handler("Librarian")`, and this method returns the exact callable function. It's a clean design: no hardcoding, no imports mid-execution, and no fragile conditionals. Just a simple lookup that tells the Executor precisely which function to execute:

```
def get_handler(self, agent_name):
    """Retrieves the function associated with an agent name."""
    handler = self.registry.get(agent_name)
    if not handler:
        raise ValueError(f"Agent '{agent_name}' not found in registry.")
    return handler
```

The final and most important method is the one designed not for code, but for the LLM itself. Here, `get_capabilities_description()` returns a structured, human-readable description of each agent's purpose, inputs, and outputs. It's the context for the LLM of the Planner. When the Planner calls the LLM to generate a plan, it includes this text so the model knows what tools are available, what each one expects as input, and what it produces as output:

```
def get_capabilities_description(self):
    """
    Returns a structured description of the agents for the Planner LLM.
    This is crucial for the Planner to understand how to use the agents.
    """

    return """
Available Agents and their required inputs:
1. AGENT: Librarian
   ROLE: Retrieves Semantic Blueprints (style/structure instructions).
   INPUTS:
   - "intent_query": (String) A descriptive phrase of the desired style or format.
   OUTPUT: The blueprint structure (JSON string).

2. AGENT: Researcher
   ROLE: Retrieves and synthesizes factual information on a topic.
   INPUTS:
   - "topic_query": (String) The subject matter to research.
   OUTPUT: Synthesized facts (String).

3. AGENT: Writer
   ROLE: Generates or rewrites content by applying a Blueprint to source material.
   INPUTS:
   - "blueprint": (String/Reference) The style instructions (usually from Librarian).
   - "facts": (String/Reference) Factual information (usually from Researcher). Use this for
   - "previous_content": (String/Reference) Existing text (usually from a prior Writer step).
   OUTPUT: The final generated text (String).
""": Final generated text
    """
```

The clarity of this description directly determines the quality of the reasoning of the Planner. A vague or incomplete description can lead to broken plans; a clear, detailed one empowers the Planner to design logical, executable workflows.

Finally, we instantiate the registry itself. This single line activates the entire system's knowledge of its agents, like flipping a switch to power the workshop's directory:

```
AGENT_TOOLKIT = AgentRegistry()
```

With this structure in place, the Planner can now reason intelligently about its team, knowing exactly who does what, what each needs to get started, and how their outputs fit together.

The Context Engine

We've met the specialists and we've cataloged them in the Agent Registry. The next step is to make the whole team think and act as one. That is the job of the Context Engine.

At its heart, the engine runs a simple but powerful loop that mirrors how people work through complex problems: plan, execute, reflect. The **Planner** thinks strategically, breaking down a broad goal into a clear plan of steps. The **Executor** gets its hands dirty, carrying out that plan, calling the right agents at the right time, and passing context along as it evolves. The **Execution Tracer** plays the quiet but essential role of observer, recording every move so we can later see not just *what* the system did, but *why* it did it.

In the sections that follow, we start with the Planner, move to the Executor, and then add the Tracer. Finally, we wire them together in `context_engine()` and walk through a worked example.

Planner

We start with the `planner` function, the strategic core of the engine. It acts as an expert project manager, translating a vague, high-level human goal into a precise, step-by-step, machine-readable JSON plan. Its intelligence comes from its ability to use an LLM as a reasoning partner.

The Planner needs two things: the user's `goal` and a `capabilities` description from the Agent Registry. With those in hand, it begins its analysis:

```
def planner(goal, capabilities):  
    """  
    Analyzes the goal and generates a structured Execution Plan using the LLM.  
    """  
  
    print("[Engine: Planner] Analyzing goal and generating execution plan...")
```

Here, `goal` is the high-level request; `capabilities` is the human-readable catalog of agents and their inputs and outputs that we return from `AgentRegistry.get_capabilities_description()`.

Next, the Planner prepares a structured briefing for the LLM. This prompt does two jobs at once. It lists the available tools, and it sets the rules for how to plan. Think of this as the Planner saying: here are the specialists you can use, here is how to reference prior results, and here is the required output format:

```

system_prompt = f"""
You are the strategic core of the Context Engine. Analyze the user's high-level goal and c

--- AVAILABLE CAPABILITIES ---
{capabilities}
--- END CAPABILITIES ---

INSTRUCTIONS:
1. The plan MUST be a JSON list of objects, where each object is a "step".
2. You MUST use Context Chaining. If a step requires input from a previous step, reference
3. Be strategic. Break down complex goals (like sequential rewriting) into distinct steps.

EXAMPLE GOAL: "Write a suspenseful story about Apollo 11."
EXAMPLE PLAN (JSON LIST):
[
    {"step": 1, "agent": "Librarian", "input": {"intent_query": "suspenseful narrative b
    {"step": 2, "agent": "Researcher", "input": {"topic_query": "Apollo 11 landing detai
    {"step": 3, "agent": "Writer", "input": {"blueprint": "$$STEP_1_OUTPUT$$", "facts":
]

EXAMPLE GOAL: "Write a technical report on Juno, then rewrite it casually."
EXAMPLE PLAN (JSON LIST):
[
    {"step": 1, "agent": "Librarian", "input": {"intent_query": "technical report struct
    {"step": 2, "agent": "Researcher", "input": {"topic_query": "Juno mission technology
    {"step": 3, "agent": "Writer", "input": {"blueprint": "$$STEP_1_OUTPUT$$", "facts":
    {"step": 4, "agent": "Librarian", "input": {"intent_query": "casual summary style"}}
    {"step": 5, "agent": "Writer", "input": {"blueprint": "$$STEP_4_OUTPUT$$", "previous
]

```

With the briefing ready, the Planner calls the LLM to act as a reasoning partner. The entire process is wrapped in a robust `try...except` block to handle potential errors, such as malformed JSON from the LLM. After calling the LLM and parsing the response, the Planner performs a critical validation. It checks if the output is a valid list. To increase resilience, it also handles cases where the LLM might wrap the list in a dictionary (e.g., `{"plan": [...]}`) and extracts the list accordingly. If the structure is still incorrect, or if any other error occurs, the `except` block logs a detailed error message, including the raw LLM output for easier debugging, and raises the exception to halt the engine. Only a successfully parsed and validated plan is returned for the Executor to use:

```

plan_json = ""
try:
    plan_json = call_llm_robust(system_prompt, goal, json_mode=True)
    plan = json.loads(plan_json)

    # Validate the output structure
    if not isinstance(plan, list):
        # Handle cases where the LLM wraps the list in a dictionary (e.g., {"plan": [...]}
        if isinstance(plan, dict) and "plan" in plan and isinstance(
            plan["plan"], list
        ):
            plan = plan["plan"]
        else:
            raise ValueError("Planner did not return a valid JSON list structure.")

    print("[Engine: Planner] Plan generated successfully.")
    return plan
except Exception as e:

```

```
print(f"[Engine: Planner] Failed to generate a valid plan. Error: {e}. Raw LLM Output:
raise e
```

This design works because it gives the LLM a clear contract to follow. Up next, we'll see how the Executor turns this plan into action.

Executor

The **Executor** is the system's on-site foreman: it runs each step, calls the right agent, and, most importantly, moves context forward so later steps can build on earlier results.

Before calling an agent, the Executor looks for placeholders in the input (things like `$$...$$`). These markers act as references to previous results, telling the system, "Use the output from step 1 here." By resolving them, the Executor transforms the plan into a connected workflow. This is context chaining in action. The `resolve_dependencies()` helper drives this process. It scans the agent's required inputs for any `$$...$$` placeholders.

This outer part of the function defines `resolve_dependencies` and creates a deep copy of the input parameters. This ensures that the original plan structure (which might be needed elsewhere) is not accidentally modified (mutated) when the placeholders are replaced:

```
def resolve_dependencies(input_params, state):
    """
    Helper function to replace $$REF$$ placeholders with actual data from the execution st
    This implements Context Chaining.
    """
    # Use copy.deepcopy to ensure the original plan structure is not modified
    resolved_input = copy.deepcopy(input_params)
```

The inner resolver (base case) defines the nested `resolve` function. Its "base case" is the logic that handles a single string value. It checks if the string is a placeholder (starts and ends with `$$`). If it is, it looks up the reference key (e.g., `STEP_1_OUTPUT`) in the state dictionary and returns the actual data:

```
# Recursive function to handle potential nested structures
def resolve(value):
    if isinstance(value, str) and value.startswith("$$") and value.endswith("$$"):
        ref_key = value[2:-2]
        if ref_key in state:
            # Retrieve the actual data (string) from the previous step's output
            print(f"[Engine: Executor] Resolved dependency {ref_key}.")
            return state[ref_key]
        else:
            raise ValueError(f"Dependency Error: Reference {ref_key} not found in exec
```

The inner resolver (recursive cases) and execution contain the recursive part of the logic. If the value is not a string placeholder, it checks if it's a dictionary or a list. If it is, it recursively calls the `resolve` function on every item inside that dictionary or list. This allows the function to find placeholders nested deep within the input. The final `return` value handles any items that are neither placeholders nor nested structures.

Finally, the outer function executes this entire process by calling

```
resolve( resolved_input ):
```

```
elif isinstance(value, dict):
    return {k: resolve(v) for k, v in value.items()}
elif isinstance(value, list):
    return [resolve(v) for v in value]
return value

return resolve(resolved_input)
```

As you can see, the Executor acts like a switchboard operator, looks up `ref_key` in the current execution state (the engine's short-term memory), and dynamically replaces the placeholder with the actual output from a previous step. This seemingly simple function is what makes a complex, stateful workflow possible.

Execution Tracer

With the Planner and Executor working in tandem, the Context Engine is functional, but not yet transparent. To close the loop, we need a way to *observe* what happens inside the system, to trace how each decision and action unfolds. The `ExecutionTrace` class, therefore, acts as the Context Engine's flight recorder, quietly documenting every stage of reasoning from the initial goal to the final output. Let's take a closer look.

When a new task begins, the Tracer is initialized with the user's `goal`. It sets up a few placeholders—`plan` for the output of the Planner, `steps` for the actions the Executor will take, and `status` to record progress. The timer starts here too, so the system can later calculate how long the whole workflow took:

```
class ExecutionTrace:
    """Logs the entire execution flow for debugging and analysis."""
    def __init__(self, goal):
        self.goal = goal
        self.plan = None
        self.steps = []
        self.status = "Initialized"
        self.final_output = None
        self.start_time = time.time()
```

Once the Planner creates its strategy, the Tracer records it using `log_plan()`. This method stores the entire JSON plan, giving us a record of what the system *intended* to do before execution began:

```
def log_plan(self, plan):
    self.plan = plan
```

As the Executor runs through the plan, `log_step()` documents each move. For every step, it captures which agent was called, what input it received (`planned_input`), what context it worked within (`resolved_context`), and what it produced (`output`). This creates a clear, chronological record of how the plan was carried out:

```
def log_step(
    self, step_num, agent, planned_input, mcp_output, resolved_input
):
    """Logs the details of a single execution step."""

    self.steps.append({
        "step": step_num,
        "agent": agent,
        "planned_input": planned_input,
        "resolved_context": resolved_input,
        "output": mcp_output['content']
    })
```

When the process completes, the Tracer wraps up with `finalize()`. It notes the final status (for example, `Success` or `Failed`), stores the final output, and calculates how long the entire task took from start to finish:

```
def finalize(self, status, final_output=None):
    self.status = status
    self.final_output = final_output
    self.duration = time.time() - self.start_time
```

Together, these methods make the Tracer the engine's meticulous scribe. It lets us reconstruct the system's reasoning at any point, whether we're debugging a failure, analyzing performance, or simply studying how the workflow unfolded. In essence, the execution Tracer gives the Context Engine something every autonomous system needs: visibility, accountability, and the quiet assurance that we can always see *how* it thinks.

Putting it all together

We now have all the building blocks in place: the Planner that thinks, the Executor that acts, and the Tracer that records every move. The final step is to connect them into a single control loop.

The `context_engine()` function is the system's ignition switch. It brings everything to life by orchestrating the complete two-phase process: first, it plans the work, and then it works the plan. Let's walk through it step by step.

When a new task starts, the function prints the goal and creates an instance of the execution Tracer, which immediately begins recording. It also loads the Agent Registry, the catalog that tells the engine which agents it has available and how to reach them. From this moment on, every action, decision, and output will be recorded in the trace.

```
def context_engine(goal):
    """
    The main entry point for the Context Engine. Manages Planning and Execution.
    """

    print(f"\n=== [Context Engine] Starting New Task ===\nGoal: {goal}\n")
    trace = ExecutionTrace(goal)
    registry = AGENT_TOOLKIT
```

Next comes planning (phase 1):

```
# Phase 1: Plan
try:
    capabilities = registry.get_capabilities_description()
    plan = planner(goal, capabilities)
    trace.log_plan(plan)
except Exception as e:
    trace.finalize("Failed during Planning")
    # Return the trace even in failure for debugging
    return None, trace
```

Here, the engine takes the user's goal and begins formulating a plan. It retrieves a full description of agent capabilities from the registry and hands it to the Planner, which uses the LLM to create a JSON plan of action. This plan breaks the goal into distinct, ordered steps, describing which agent should do what and in what sequence. Once the Planner finishes, the plan is logged by the Tracer, preserving the strategy before execution begins. If anything goes wrong at this stage, say, a formatting error or planning failure, the system records it cleanly and exits with the trace intact.

With a plan in place, the engine transitions into execution (phase 1):

```
print(f"\n[Engine: Executor] Starting Step {step_num}: {agent_name}")

try:
    handler = registry.get_handler(agent_name)

    # Context Assembly: Resolve dependencies
    resolved_input = resolve_dependencies(planned_input, state)

    # Execute Agent via MCP
    # Create an MCP message with the RESOLVED input for the agent
    mcp_resolved_input = create_mcp_message(
        "Engine", resolved_input)
    mcp_output = handler(mcp_resolved_input)

    # Update State and Log Trace
    output_data = mcp_output["content"]

    # Store the output data (the context itself)
    state[f"STEP_{step_num}_OUTPUT"] = output_data
    trace.log_step(step_num, agent_name, planned_input,
        mcp_output, resolved_input)
    print(f"[Engine: Executor] Step {step_num} completed.")

except Exception as e:
    error_message = f"Execution failed at step {step_num}({agent_name}):{e}"
    print(f"[Engine: Executor] ERROR: {error_message}")
    trace.finalize(f"Failed at Step {step_num}")
    # Return the trace for debugging the failure
    return None, trace
```

This loop is where the Context Engine actually does the work. The Executor iterates through each step of the plan, calling the appropriate agent for the job. Before it invokes an agent, it uses `resolve_dependencies()` to check for any placeholders in the input (things like references to outputs from earlier steps) and replaces them with the actual data stored in the current execution state. The inputs are then wrapped into an MCP message and sent to the agent, whose response becomes the output for that step. Each output is stored in the state dictionary, creating a

short-term memory that future steps can draw from. The Tracer logs every detail along the way: which agent was called, what input it received, how the context was resolved, and what result was produced.

Once all steps have run, the engine finalizes the process:

```
final_output = state.get(f"STEP_{len(plan)}_OUTPUT")
trace.finalize("Success", final_output)
print("\n=== [Context Engine] Task Complete ===")
return final_output, trace
```

At this point, the final output from the last step is retrieved and recorded. The Tracer wraps up by marking the run as successful, logging the result, and measuring the total duration from start to finish. The function then returns two things: the generated output and the full execution trace.

Running the engine

We've built each piece of the puzzle: the specialists, the registry, the Planner, the Executor, and the Tracer. After all this architectural work, it's time to turn the key and hear the engine run. Let's see what happens when the Context Engine is asked to tackle a creative goal from start to finish. We'll start with a simple prompt and ask it for a suspenseful children's story about Apollo 11:

```
print("***** Example 1: STANDARD WORKFLOW (Suspenseful Narrative) *****\n")
goal_1 = "Write a short, suspenseful scene for a children's story about the Apollo 11 moon

# Run the Context Engine
# Ensure the Pinecone index is populated (from Ch3 notebook) for this to work.
result_1, trace_1 = context_engine(goal_1)

if result_1:
    print("\n***** FINAL OUTPUT 1 *****\n")
    display(Markdown(result_1))
    print("\n\n" + "="*50 + "\n\n")
    # Optional: Display the trace to see the engine's process
    # trace_1.display_trace()
```

When this goal is passed to the engine, the Planner springs into action:

1. It reasons that to create a "suspenseful" scene that also highlights "danger," it first needs a stylistic guide. Therefore, its first planned step is to call the Librarian to retrieve a blueprint for suspense.
2. Next, it recognizes the factual component, "Apollo 11 moon landing", and knows it needs accurate details. Its second step is to task the Researcher with fetching relevant facts.
3. Finally, with a plan for both style and substance, it creates the third step: call the Writer to generate the final content, combining the blueprint from step 1 with the facts from step 2.

The Executor then follows this logical plan:

1. The `ExecutionTrace` class records the plan, each step, inputs, outputs, and final result
2. The `planner` function generates a JSON execution plan based on the user's goal and agent capabilities
3. The Executor runs the plan, resolving dependencies and calling agents

4. The `resolve_dependencies()` helper replaces placeholders with actual outputs from previous steps

The output provides the full traceability of the Context Engine we expect. It first displays the title and goal of the task:

```
***** Example 1: STANDARD WORKFLOW (Suspenseful Narrative) *****
=== [Context Engine] Starting New Task ===
Goal: Write a short, suspenseful scene for a children's story about the
Apollo 11 moon landing, highlighting the danger.
```

The system then provides the administrator and user of the system with the trace of the flow of the task. This is now a fundamental requirement in strategic corporate projects, as well as a legal constraint in some regions in the world (the European AI Act, for example). In any case, for a strategic decision-making process, it is best practice to be able to explain how the system reached a result, as we do here:

```
[Engine: Planner] Analyzing goal and generating execution plan...
[Engine: Planner] Plan generated successfully.

[Engine: Executor] Starting Step 1: Librarian

[Librarian] Activated. Analyzing intent...
[Librarian] Found blueprint 'blueprint_suspense_narrative' (Score: 0.66)
[Engine: Executor] Step 1 completed.

[Engine: Executor] Starting Step 2: Researcher

[Researcher] Activated. Investigating topic...
[Researcher] Found 2 relevant chunks. Synthesizing...
[Engine: Executor] Step 2 completed.

[Engine: Executor] Starting Step 3: Writer
[Engine: Executor] Resolved dependency STEP_1_OUTPUT.
[Engine: Executor] Resolved dependency STEP_2_OUTPUT.

[Writer] Activated. Applying blueprint to source material...
[Engine: Executor] Step 3 completed.

=== [Context Engine] Task Complete ===
```

Finally, the engine displays the requested text:

```
***** FINAL OUTPUT 1 *****
July 20, 1969. Apollo 11. NASA's Apollo program. The Space Race hums in the background lik
Three of us. Neil Armstrong. Buzz Aldrin. Michael Collins.
The Moon fills the window. Black sky. Hard light. Long shadows.
Numbers creep. The guidance drifts. The margin thins. Fuel is low.
I take manual control of the Lunar Module Eagle. The stick is firm. My glove is steady. Th
The cabin is small. A soft hiss in my ears. Breathing. Fans. A tremor underfoot. Shadows c
We descend. Slow. Slower. The surface slides by, gray and strange. The target shifts. I nu
The world listens. Broadcast live. A billion quiet rooms. A billion held breaths.
The hum rises, falls. The Eagle steadies. The scene tightens to a pin.
Then—stillness.
Silence folds in. My heartbeat steps out of time and back again. The Moon waits, wide and
I look down at that pale ground. Distance, then inches. I move carefully. Deliberate. Ever
The hatch. The edge. The black beyond the light. I feel the weight change. Not heavy. Not
A boot hovers. I watch the shadow first. It kisses the surface before I do.
Then contact. A first step. On the Moon.
```

The world is there with me, far away and everywhere at once. The air in my suit whispers.

=====

In production, you can decide to only display the trace of execution to the administrators or save the output in an execution log. Or, you can decide that, during the training phase, the users can also see the execution trace to understand the Context Engine. This can be decided in workshops during the implementation.

With this successful execution, we can now step back and appreciate what we've built. Every architectural component is working in concert, transforming abstract intent into structured, traceable, and meaningful output. Let's reflect on this architectural leap and the design principles that make it work so elegantly in practice.

Summary

The core challenge addressed in this chapter was the rigidity of predefined workflows. The goal was to create an advanced controller that can dynamically plan and execute complex tasks by reasoning about the tools at its disposal. The journey began with building the Planner, which acts as the strategic core, creating a step-by-step recipe to achieve a goal. We then constructed the Executor, the operational manager that carries out the plan. Finally, a Tracer was included to ensure the entire process is transparent and debuggable.

The key innovation introduced was the planning phase. Here, the Planner consults an Agent Registry, a toolkit describing the capabilities of each specialist agent (Librarian, Researcher, Writer). Armed with this knowledge, the Planner uses an external LLM to generate a custom, multi-step JSON plan tailored to the user's specific high-level goal. This separates the *what to do* from the *how to do it*. The Executor then follows the JSON plan, calling the specialist agents in the correct sequence. This phase implements context chaining, which was discussed in [Chapter 1](#). This allows the system to handle complex, dependent tasks that a linear workflow could not.

The chapter culminated in a fully functional Context Engine that can think for itself. By dynamically generating its own workflows, the system is no longer a simple assembly line but an intelligent and modular workshop. In the next chapter, we will harden this engine for real-world applications by integrating production-level features such as context management, security guardrails, and error handling.

Questions

1. Does the executor create the strategic, step-by-step plan to achieve the user's goal? (Yes or no)
2. Is the agent registry's main purpose to log every action for debugging? (Yes or no)
3. Does the text describe context chaining as the process where one agent's output becomes the input for the next agent? (Yes or no)
4. Are the specialist agents (Librarian, Researcher, and Writer) represented by the blue color in the architectural diagram? (Yes or no)
5. Does the `resolve_dependencies()` function allow agents to query the vector DB? (Yes or no)

6. Does the planner use the `get_capabilities_description()` method from the agent registry to understand what tools are available? (Yes or no)
7. Is the `context_engine()` function the primary interface to the external LLM? (Yes or no)
8. Does the Writer agent handle procedural RAG to fetch stylistic blueprints? (Yes or no)
9. Is the entire process bookended by orange components representing the user's goal and the final output? (Yes or no)
10. Do all agents communicate using the **Model Context Protocol (MCP)**? (Yes or no)

References

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*.
- Yao, S., Zhao, J., Yu, D., Du, N., Sha, H., & Tsvetkov, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629**arXiv preprint arXiv:2210.03629*.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761**arXiv preprint arXiv:2302.04761*.

Further reading

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, J., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns—*AI Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



Table of contents collapsed